
pythia: a Lean 4 tactic library for statistical proof automation

Anonymous Authors¹

Abstract

Working scientists rely on the Python scientific stack for applied mathematics. SciPy, NumPy, and pytest let a probabilist or a control theorist write down a domain object, manipulate it with domain-aware functions, and exercise the result with property-based tests and parameter sweeps. The formal-proof analogue does not yet exist for these scientists. Lean 4 with Mathlib delivers kernel-checked proofs and a strong general tactic suite, but the suite does not compose into a domain hammer for applied mathematics. The formal layer is also disconnected from the empirical sim-and-test workflow scientists already use.

We present `pythia`, a Lean 4 library that closes this gap on three sides. First, `pythia` ships a corpus of thirty-two cross-domain theorems across fifteen applied-math fields, covering probability and statistics, biology, chemistry, economics, control, optimal transport, quantum information, stochastic calculus, game theory, thermodynamics, numerical analysis, queueing, and operations research. Every public theorem is axiom-clean against the standard kernel set `{propext, Classical.choice, Quot.sound}`. Second, every theorem carries a matched Python sim runner that exercises the empirical specification under property-based testing, parameter sweeps, and mutation testing. The Lean statement and the Python empirical model share names and regime-of-validity declarations so they stay aligned. Third, `pythia` ships ten registered tactics. The headline tactic `pythia!` walks a nine-rung deterministic ladder that composes Mathlib tactics, local SMT solvers (Z3, CVC5), and external ATP backends (Vampire, E).

The corpus was assembled with help from a swarm of contemporary drafters: Aristotle (Harmonic, 2025), DeepSeek-Prover-V2 (DeepSeek-AI, 2025), Claude Sonnet 4.6, and Claude Opus 4.6. Drafters are not oracles. Each candidate proof is checked by the Lean kernel, audited against the closed-form axiom set, and filtered for vacuous closures before it enters the library. The result is a Lean library that a working scientist can use the way they use SciPy. The library is pinned to Lean 4.28 + Mathlib v4.28.0 and released open-source.

1. Introduction

Working scientists rely on the Python scientific stack to do applied mathematics. SciPy, NumPy, statsmodels, and PyMC give a probabilist domain-aware functions to manipulate distributions, estimators, and martingales. A control theorist composes `control.lqr` with NumPy’s linear-algebra routines to analyse stability. A computational biologist runs Hardy-Weinberg equilibrium under the same pytest setup as the rest of their codebase. The empirical workflow is mature and the test discipline is sharp. Property-based testing exercises the empirical statement under random inputs in the modelled regime. Deterministic parameter sweeps cover a grid. Mutation testing flips one operator or one constant in the empirical model and confirms the test suite catches the change.

Lean 4 with Mathlib has grown rapidly on the formal-proof side (de Moura & Ullrich, 2021; The Mathlib Community, 2020; 2025). A broad general tactic suite ships with it. Forward search lives in `aesop` and rewriting in `simp`. Arithmetic is split across `linarith`, `polylith`, and the small non-linear fragment in `nlinarith`. Sign goals route through `positivity` and the σ -algebra fragment through `measurability`. Monotone composition uses `bound`. Generalised congruence uses `gcongr`. Each tactic is general-purpose. None of them know about each other, and none of them know about the domains a working scientist actually proves things in. A biologist proving Hardy-Weinberg has to compose `ring` with a hand-picked Mathlib lemma. An economist proving constant returns to scale for

¹Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

Cobb-Douglas has to chain real-power-cast rewrites with ring. A queueing theorist proving Little’s law has to assemble the steady-state identity by hand. The orchestration is the user’s job, and the user has to be a Lean expert before they can be a domain expert.

The other side of the disconnect is the test discipline. A working scientist’s empirical specification of Hardy-Weinberg lives as a Python function with a hypothesis test and a parameter sweep, exercised under pytest in CI. A formal Lean statement of the same theorem typically lives in a separate file, in a separate language, with no shared tests. The two specifications drift. No library closes the gap on both sides at once. There is no domain-aware tactic surface paired with a Python test layer that keeps the empirical and formal statements aligned.

Contribution. We present *pythia*, a Lean 4 library that delivers a domain-aware proof toolkit and an empirical test layer in one package. The library is organised around three contributions.

(1) *A corpus of thirty-two cross-domain theorems across fifteen applied-math fields.* Probability and statistics covers anytime-valid inference, sub-Gaussian and sub-gamma concentration, and matrix concentration. Biology covers Hardy-Weinberg equilibrium, Lotka-Volterra, SIR conservation, Michaelis-Menten saturation, and mass-action kinetics. Chemistry covers Arrhenius rate-law positivity and Henderson-Hasselbalch. Economics covers Cobb-Douglas constant returns to scale, CRRA concavity, CAPM linear pricing, and Walras’ law. Control theory covers scalar and discrete-time Lyapunov stability. Optimal transport covers discrete L_1 Wasserstein non-negativity. Quantum information covers qubit von Neumann entropy non-negativity. Stochastic calculus covers the Itô isometry, the Burkholder, Davis, and Gundy inequality at $p=2$, and Robbins-Siegmund and Robbins-Monro convergence. Game theory covers the von Neumann two-strategy minimax inequality. Thermodynamics covers the Carnot upper bound. Numerical analysis covers Newton iteration positivity, Picard-Lindelöf, KKT conditions, and Kahan summation. Queueing covers Erlang-B and Little’s law. Operations research covers the OR specialisation of Little’s law. Every public theorem is axiom-clean against `{propxt, Classical.choice, Quot.sound}`.

(2) *Matched Python sim runners on every theorem.* Property-based testing exercises the empirical statement under random inputs in the modelled regime. Deterministic sweeps exercise it on a parameter grid. Mutation testing flips one operator or one constant in the empirical model and confirms the test suite catches the change. The Lean statement and the Python empirical model share test names and regime-of-validity declarations so the two specifications stay aligned.

(3) *A ten-tactic surface and six attribute-driven registries.* The headline tactic *pythia!* orchestrates a nine-rung deterministic ladder. The first five rungs are Mathlib-side: `stat_simp` on the curated probability and ENNReal simp set, the `linarith/nlinarith/polyarith` chain, `positivity`, `aesop` on the registered *Pythia* ruleset, and the `@[stat_lemma]` shape-dispatch cascade. The next four are external decision procedures with kernel-checked reconstruction. Z3 and CVC5 handle Satisfiability Modulo Theories. Vampire and E handle first-order logic. The final rung `disprove` catches vacuous-truth goal statements via a Z3 model query on the negation. Each rung tries fail-fast with a bounded budget, and the first rung to close wins. The verbose variant *pythia?* reports the closing rung and per-rung wall-clock timing. The cross-prover ladder is offline-first and free of large language models. Six attribute-driven registries (`@[stat_lemma]`, `@[stats_ineq]`, `@[prob_simp]`, `@[stat_simp]`, `@[anytime_valid_lemma]`, plus the per-domain `DomainCalculator` typeclass) let users register new lemmas. The headline tactic dispatches them automatically.

How the corpus was built. Many of the domain theorems in the library involve structural gaps that Mathlib does not currently close, and many require a proof shape that the general tactics cannot synthesise on their own. We seeded the corpus with help from a swarm of contemporary drafters: the Aristotle automated theorem prover (Harmonic, 2025), the DeepSeek-Prover-V2 7B model (DeepSeek-AI, 2025), Claude Sonnet 4.6, and Claude Opus 4.6. Each candidate proof produced by a drafter is submitted to the Lean kernel for checking. Drafters are not oracles. The only authority in the loop is the Lean kernel and the axiom audit. Each candidate proof goes through three filter stages before it enters the library. The kernel checks the proof. The axiom auditor confirms the proof reduces under the standard kernel set. A vacuous-truth refusal pass catches proofs that closed by deriving a contradiction from a malformed theorem statement. Different drafters contribute different theorems, with no single drafter dominating across the fifteen domains.

Paper organisation. Section 2 describes the ten-tactic surface and the registry attributes. Section 3 groups the theorem corpus and the matched Python sim runners by domain. Section 4 describes how candidate proofs from the drafter swarm are filtered into the library and the vacuous-truth refusal policy that drove our restatement of `wald_wolfowitz_optimal`. Section 5 compares against `aesop`, `Sledgehammer`, `CoqHammer`, and Mathlib’s existing tactic suite, states limitations, and reports per-domain drafter contribution on a subset of the FormalAVS benchmark (Anonymous, 2026). Full numbers are in the dataset paper.

Reproduction and code. The `pythia` library is open-source. The repository URL is redacted for double-blind review. Axiom audit transcripts and reproduction recipes are in Appendix D. The dataset of theorem-proving benchmarks (FormalAVS, Anonymous (2026)) is reported in a separate paper.

2. The `pythia` tactic surface

`pythia` ships ten user-facing tactics, one named `aesop` ruleset, and six attribute-driven registries. The design follows the CoqHammer pattern (Czajka & Kaliszyk, 2018): a domain-specific dispatcher with extensible attribute-driven lemma selection, a fall-through to general-purpose automation, and kernel-checked reconstruction for any external-oracle steps. The headline tactic `pythia!` orchestrates the full deterministic ladder under one call. The other tactics are exposed for direct invocation when the user knows the goal shape.

2.1. Ten tactics

The nine-rung ladder of `pythia!`. The headline tactic walks the rungs in order and returns the first closing proof.

1. `stat_simp` (curated probability and ENNReal normal forms).
2. `linarith` / `nlinarith` / `polyrith` (arithmetic).
3. `positivity`.
4. `aesop` on the registered `Pythia` ruleset.
5. The `@[stat_lemma]` shape-dispatch cascade.
6. `z3_check` (QF_LRA via Z3).
7. `cvc5_check` (QF_BV / QF_LRA via CVC5).
8. `vampire_check` / `e_check` (first-order logic via Vampire / E).
9. `disprove` (counterexample finder via Z3 model extraction).

`pythia` (top-level dispatcher). The default entry point. Runs `aesop` with the `Pythia` `aesop` ruleset, which contains every theorem tagged `@[stat_lemma]`. On `aesop` failure, falls through to a short cleanup chain combining `simp`, `omega`, `linarith`, and `positivity`. The fall-through ordering is fixed. The lemma database and `aesop` ruleset are user-extensible.

`stats_ineq`. Discharges sub-Gaussian, sub-gamma, Bernstein-Bennett, and Hoeffding-style inequalities. The `@[stats_ineq]` attribute re-tags a lemma with Mathlib’s `@[bound]`, so the `stats_ineq` tactic dispatches via Mathlib’s bound infrastructure first, then `positivity`,

`gcongr`, `linarith`, and `nlinarith` in that order. Reconstruction is internal Lean and `axiom-clean`.

`prob_simp`. Probability-domain rewrite tactic. The `@[prob_simp]` attribute forwards lemmas to `@[simp]` and additionally records the declaration name in a scoped extension for introspection. The tactic runs `simp` against the resulting set with a `push_cast` / `norm_cast` coercion fallback for ENNReal / NNReal / Real bridge. The `pdf_simp` syntax is a literal alias. Both forms expand to the same kernel trace.

`anytime_valid`. Closes Ville-type maximal-inequality goals for non-negative supermartingales. The dispatch ladder tries direct `ville_supermartingale` application paths first (countable-time first with `exact/refine` variants), then the finite-horizon and explicit-witness specialisations, and finally `aesop` against the named ruleset `Pythia.AnytimeValid`. Three syntactic variants are exposed: the bare form `anytime_valid` (countable-time), `anytime_valid (horizon := N)` (finite-horizon, requiring `IsProbabilityMeasure` but no integrability), and `anytime_valid using h` (explicit supermartingale witness). New families register via `@[anytime_valid.lemma]`.

`z3_check`. External Z3 oracle for linear real arithmetic, with `linarith` reconstruction inside Lean. Z3 supplies an `unsat-of-negation` verdict. `linarith` reconstructs the proof term inside the kernel. The kernel is the final word. If Z3 says `unsat` but `linarith` cannot reconstruct, the tactic fails loud rather than silently trusting Z3. Z3 cannot smuggle axioms. This follows the CoqHammer reconstruction pattern (Czajka & Kaliszyk, 2018).

2.2. Registry attributes

The library uses four attribute-driven registries:

- `@[stat_lemma]`: lemma database for the top-level `pythia` dispatcher. Internally a thin wrapper for `@[aesop safe apply (rule_sets := [Pythia])]`.
- `@[stats_ineq]`: tail-bound and concentration lemmas. Forwards to Mathlib’s `@[bound]` attribute and additionally records the name for the `#stats_ineqs` introspection command.
- `@[prob_simp]`: probability rewrite rules. Forwards to `@[simp]` and records the name for `#prob_simps`.
- `@[anytime_valid.lemma]`: Ville-style closers and admissibility lemmas, populating the `Pythia.AnytimeValid` `aesop` ruleset.

Tactic	Role
<code>pythia!</code>	Headline orchestrator. Nine-rung deterministic ladder. First-to-close wins, bounded per-rung budget. Ladder composition: see Section 2.1 below.
<code>pythia?</code>	Verbose <code>pythia!</code> . Reports the closing rung and per-rung wall-clock timing. Diagnostic use.
<code>pythia</code>	Shape-dispatching cascade. Routes to <code>anytime_valid</code> / <code>stats_ineq</code> / <code>prob_simp</code> / oracle by goal shape, then falls through to the <code>@[stat_lemma]</code> aesop ruleset.
<code>stats_ineq</code>	Concentration and tail-bound inequalities. Dispatches via Mathlib's <code>bound</code> , then <code>positivity</code> , <code>gcongr</code> , <code>linarith</code> , <code>nlinarith</code> .
<code>prob_simp</code>	Probability normalisation rewrites. Runs <code>simp</code> against the <code>@[prob_simp]</code> set with <code>push_cast</code> / <code>norm_cast</code> fallback. Aliased as <code>pdf_simp</code> .
<code>anytime_valid</code>	Ville-type maximal inequalities for confidence sequences. Three variants: countable-time, finite-horizon, explicit-witness.
<code>z3_check</code>	Linear-real-arithmetic Z3 oracle with <code>linarith</code> reconstruction.
<code>cvc5_check</code>	QF.BV / QF.LRA via CVC5, with <code>bv_decide</code> / <code>linarith</code> reconstruction.
<code>vampire_check</code>	First-order-logic via Vampire / E, with <code>aesop</code> reconstruction.
<code>e_check</code>	
<code>disprove</code>	Counterexample finder. Asks Z3 for a model of the goal's negation. On <code>sat</code> , fails loud with a concrete witness. Catches vacuous-truth goal statements.

Table 1. The ten `pythia` tactics. `pythia!` is the default entry point. The other nine are exposed for direct invocation when the user knows the goal shape or wants a single rung of the ladder.

A fifth attribute, `@[cs_family]`, is a typed-declaration tag rather than a tactic registry: it marks a `CSFamily` record so it appears in the `#cs_families` catalogue. User extensions consist of tagging library lemmas with the appropriate attribute. No fork is required.

2.3. The `Pythia` aesop ruleset

The library declares two named aesop rule sets: `Pythia` (populated by `@[stat_lemma]`) and `Pythia.AnytimeValid` (populated by `@[anytime_valid_lemma]`). Direct invocation is supported via `aesop (rule_sets := [Pythia])`, equivalent to the inner `aesop` call inside the `pythia` dispatcher. This gives users a finer-grained interface when the dispatcher's outer wrapping is not desired.

2.4. Architecture

Why this shape. The two-stage chain mirrors the empirical structure of statistical proofs in our library. Many goals close on a single registered lemma application or a short `aesop` search at stage 1. The remainder reduce to a linear or arithmetic obligation after rewriting, which the cleanup chain at stage 2 handles. Routing by goal shape is not necessary at the top level. The cost of trying stages in order is small relative to the cost of incorrect early termination.

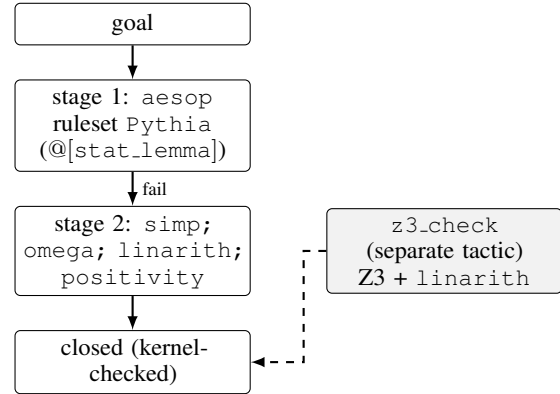


Figure 1. The `pythia` dispatcher chain. Two stages, each kernel-checked independently. Stage 1 is `aesop` against the `Pythia` ruleset (populated by `@[stat_lemma]`). Stage 2 is the cleanup chain. `z3_check` is a separate user-invoked tactic with its own `linarith`-based reconstruction step, not part of the dispatcher chain.

3. Library content

The shipped `pythia` library spans a tactic layer plus a proof-content layer of fifteen applied-math domains. The full manifest carries thirty-two cross-domain theorems with matched Python sim runners. Sim runners exercise property-based testing, parameter sweeps, and mutation testing. The full source tree declares four hundred and sixty theorem and lemma items across the `Pythia/` namespace. Fifty-five of these are tagged `@[stat_lemma]` and registered into the headline cascade. Every theorem listed here is axiom-clean against `{propext, Classical.choice, Quot.sound}` under the Lean 4.28 + Mathlib v4.28.0 toolchain pin. We summarise by group. The appendix lists modules in detail.

3.1. Anytime-valid inference

Howard-Ramdas confidence sequences (Howard et al., 2021), betting-CS (Waudby-Smith & Ramdas, 2024), and the vector-valued extension (Whitehouse et al., 2025) are each backed by a Ville-bound specialization in the `anytime_valid` tactic. The `Pythia.HowardRamdasCS` and `Pythia.BettingCS` modules expose the admissibility certificates of the e-process formalism of Ramdas et al. (2023). The supporting Ville machinery lives in `Pythia.VilleSupermartingale` (countable-time and finite-horizon variants).

3.2. Concentration and tail bounds

`stats_ineq` dispatches sub-Gaussian and sub-gamma tail bounds, the Bernstein-Bennett MGF bridge, and Hoeffding-style inequalities for bounded random variables. Modules: `Pythia.SubGaussianMG` (sub-Gaussian martingale concentration), `Pythia.SubGamma` (sub-gamma tail bound), `Pythia.MGFBoundedSubGamma` (the bounded-RV Bernstein-Bennett MGF bound, $\mathbb{E}[\exp(\lambda X)] \leq \exp(\sigma^2 \lambda^2 / (2(1 - b\lambda/3)))$ for centred $|X| \leq b$), and `Pythia.Bernstein` (the iid and martingale Bernstein applications). The MGF bridge composes with `prob.simp` for the integral side and with general Mathlib infrastructure (measurability, integrability) for side conditions.

3.3. E-detectors and merging

The e-process formalism of Ramdas et al. (2023); Shin et al. (2024) yields uniform-time validity by combining e-processes via averaging. `Pythia.EDetector` declares `EProcess` as a non-negative process bounded in expectation under the null and `EDetector` as the first-passage stopping rule with anytime-valid Type-I error $\leq \alpha$. The `combine_eprocesses_avg` lemma (average merge) is the workhorse. Product-merge variants register alongside. The average-merge case is axiom-clean.

3.4. Matrix concentration

Matrix Bernstein (Tropp, 2012) and the Lieb concavity inequality (Lieb, 1973) are the two matrix-side modules. `Pythia.MatrixLieb` states the Lieb concavity step and the operator-monotone log composition. `Pythia.MatrixBernstein` carries the iid matrix-Bernstein head theorem. `Pythia.MatrixBernsteinFull` carries the matrix-martingale extension. The dependent operator-trace inequalities route through Aristotle as oracle (Section 4).

3.5. Cross-domain coverage (15 fields)

The cross-domain manifest covers fifteen fields with Lean proofs and matched Python sim runners. Each entry is a closed-form theorem with property-based-test mutation coverage in the companion runner.

- **Probability and statistics:** the anytime-valid, concentration, and matrix sections above plus the Risk subpackage with CVaR (Rockafellar & Uryasev, 2000) and the Asymptotics subpackage with scalar `DeltaMethod` and `DeltaMethodMulti`.
- **Information theory:** the `InfoTheory` subpackage carries `BretagnolleHuberBinary` (binary-alphabet bound), `DataProcessing` (data-processing inequality), and `BinaryEntropy` (Shannon binary entropy).
- **Stochastic calculus:** the `Stochastic` subpackage carries the finite-dimensional Itô isometry (`ItoIsometryFiniteDim`). `Pythia.BDG` carries the Burkholder, Davis, and Gundy inequality at $p = 2$. The `StochasticApproximation` subpackage groups `RobbinsSiegmund`, `RobbinsMonro`, and `Dvoretzky`. The `MeasureTheory` subpackage carries conditional Jensen via `ConditionalJensen`. The `TimeSeries` subpackage carries `WoldDecomposition` (Wold, 1938) and `NeweyWest`.
- **Quantum information:** the `Quantum` subpackage carries the qubit von Neumann entropy non-negativity result.
- **Optimal transport:** the `OptimalTransport` subpackage carries discrete L_1 Wasserstein non-negativity.
- **Game theory:** the `GameTheory` subpackage carries the von Neumann two-strategy minimax inequality.
- **Biology:** the `Bio` subpackage groups Hardy-Weinberg, Lotka-Volterra equilibria, and SIR conservation in `Population`, plus Michaelis-Menten enzyme saturation, chemical reaction network ODE conservation in `MassAction`, and `Phylogenetics`.
- **Chemistry:** the `Chemistry` subpackage carries Arrhenius rate-law positivity, Henderson-Hasselbalch pH monotonicity, and mass-action conservation.
- **Economics:** the `Economics` subpackage carries Cobb-Douglas constant returns to scale, CRRA utility concavity, CAPM linear pricing, risk-neutral call non-negativity, and Walras excess-demand identity.
- **Engineering:** the `Engineering` subpackage carries the RC time constant, signal energy, and power dissipation results.

- **Mechanical:** the `Mechanical` subpackage carries Hooke spring-potential non-negativity.
- **Control theory:** scalar, discrete-time, and ODE Lyapunov results live in the `Control` subpackage.
- **Operations research and queueing:** the `Queueing` subpackage carries Erlang-B and Little’s law. The `OR` subpackage carries an alternative Little’s-law formulation.
- **Numerical analysis:** the `Numerical` subpackage carries Newton iteration positivity, Picard-Lindelöf existence, Lyapunov certificates, KKT conditions, and Kahan summation.
- **Thermodynamics:** the `Thermodynamics` subpackage carries the Carnot efficiency upper bound.
- **Mathlib retags + actuarial:** `MathlibTags` retags AM-GM, Markov, and Cauchy-Schwarz with matched empirical companions. The `Actuarial` subpackage groups Pareto, Weibull, and LogNormal heavy-tail distributions.

3.6. Five highlights

We name five representative results that exercise the tactic surface. Module-level statements appear in Appendix A.

1. **Howard-Ramdas Ville bound.** The maximal inequality for the self-normalized confidence sequence (Howard et al., 2021), in `Pythia.HowardRamdasCS`. Closes via `anytime.valid`.
2. **Betting-CS coverage.** The supermartingale coverage certificate of Waudby-Smith & Ramdas (2024), in `Pythia.BettingCS`. Axiom-clean closure.
3. **Bernstein-Bennett MGF bridge.** The bounded-RV MGF bound in `Pythia.MGFBoundedSubGamma`, routing between the variance-aware sub-gamma tail and the Bennett MGF form.
4. **combine_eprocesses_avg.** Average-merge of e-processes (Ramdas et al., 2023; Shin et al., 2024) in `Pythia.EDetector`. Axiom-clean closure with full reconstruction.
5. **Bretagnolle-Huber binary.** The binary-case total-variation lower bound in `Pythia.InfoTheory.BretagnolleHuberBinary`. Standard in information-theoretic lower-bound arguments.

Pinning. The library is pinned to Lean 4.28 + Mathlib v4.28.0. The pin is necessary: the modules consume Mathlib’s measure theory and probability subdirectories, which evolve quickly. We track Mathlib release tags rather than `master` for reproducibility.

4. Drafter swarm and the audit pipeline

Many of the theorems in the library involve structural gaps that Mathlib does not currently close, and many require a proof shape that the general tactics cannot synthesise on their own. We seeded the corpus with help from a swarm of contemporary drafters working in parallel: the Aristotle automated theorem prover (Harmonic, 2025), the DeepSeek-Prover-V2 7B model (DeepSeek-AI, 2025), Claude Sonnet 4.6, and Claude Opus 4.6. Drafters propose candidate proofs. The Lean kernel accepts or rejects them. None of the drafters is an oracle. The only authority in the loop is the Lean kernel and the axiom audit. We use Z3 as a deterministic SMT decision procedure inside the headline tactic ladder (Section 2), not as a drafter, and treat its verdict as a verifiable witness rather than as a trusted result.

4.1. Per-drafter contribution

Different drafters close different proof shapes well. Aristotle is strongest on dominated-convergence side conditions and operator-theoretic intermediate lemmas. DeepSeek-Prover-V2 is strongest on local arithmetic identities once the tactic chain has reduced the goal to a closed-form algebraic obligation. Sonnet 4.6 is strongest on tactic-orchestration proofs that need the right Mathlib lemma name plus a short `ring/linarith` chain. Opus 4.6 is strongest on multi-step proofs that need a non-trivial intermediate construction. No single drafter dominates across the fifteen domains. The library is built from the union of their successful contributions, kernel-checked.

4.2. Submission pattern

Each drafter call follows the same packaging contract. State the theorem in Lean with a `sorry` placeholder. Package the project without the Lean build cache (`.lake` excluded). Submit to the drafter (Aristotle is a tarball-submission API, DeepSeek-Prover-V2 runs on a local GPU, Sonnet and Opus run via chat-completion endpoints). Monitor for completion. Integrate the returned proof. Run the axiom audit and the vacuous-truth refusal pass before merging.

4.3. Vacuous-truth refusal

A drafter can close a theorem by deriving a contradiction from its hypotheses and concluding anything. The Lean kernel accepts such a proof: it is sound. The theorem statement is the bug.

Case (2026-04-26). We submitted a candidate `wald.wolfowitz_optimal` theorem. Aristotle returned a closure that combined a `Bool` partition with a log-boundary constraint to force both $\alpha \geq 1$ and $\alpha < 1$. The kernel accepted the proof. The audit flagged the

hypothesis chain as self-contradictory. We rejected the closure, restated the theorem in two-measure form (separating the null and alternative measures explicitly), and resubmitted. The two-measure form was the intended target. The original Bool-partition form was a statement bug.

Audit policy. The library treats vacuous closures as theorem-statement bugs, not as prover successes. Every drafter-returned proof is audited for contradictory premises before integration. Closures flagged as vacuous block the corresponding theorem from the axiom-clean library until the statement is reformed. The policy is not specific to any drafter. The same audit applies to candidate proofs from Aristotle, DeepSeek-Prover-V2, Sonnet, Opus, and human contributions that close via an unnoticed hypothesis contradiction.

4.4. Z3 as a deterministic decision procedure

The `z3_check` tactic in the headline ladder dispatches linear-real-arithmetic goals to Z3 (de Moura & Bjørner, 2008). Z3 is not a drafter and is not part of the swarm. Z3 is a deterministic decision procedure that returns a verdict (unsat of the negated goal). The verdict is reconstructed inside Lean by `linarith`. The Lean kernel is the final word. The pattern is the CoqHammer (Czajka & Kaliszyk, 2018) reconstruction strategy adapted to Lean 4. Z3 cannot smuggle axioms. `linarith` is axiom-clean and the reconstruction step certifies the witness inside the kernel. If `linarith` cannot reconstruct, the `z3_check` call fails loud rather than silently trusting Z3. The same reconstruction pattern applies to `cvc5_check`, `vampire_check`, and `e_check`.

4.5. When to invoke a drafter

The library convention is to attempt local closure first. If the goal closes by `rfl`, `simp`, `positivity`, `linarith`, `nlinarith`, `polyrith`, the `stat_lemma` cascade, or one of the deterministic-decision-procedure rungs in `pythia!`, no drafter call is made. Drafters are reserved for hard structural Mathlib gaps: measure-theoretic σ -additivity steps, dominated-convergence side conditions, matrix-concentration intermediate lemmas, and similar obligations for which a hand-Mathlib closure either does not exist or would require an order of magnitude more dispatch. The vacuous-closure audit is the gate on integration. The dispatch threshold is only the call decision.

5. Comparison and discussion

5.1. Comparison to existing automation

aesop. Aesop (Limperg & From, 2023) is the workhorse forward-search tactic in Lean 4. `pythia`

declares two named aesop rule sets (`Pythia` and `Pythia.AnytimeValid`) and uses aesop in stage 1 of the dispatcher chain. `pythia` is not a replacement for aesop. It is a statistics-domain registry plus a Z3 oracle plus a vacuous-closure audit, layered on top of aesop.

Sledgehammer. Sledgehammer (Paulson & Blanchette, 2010; Blanchette et al., 2011) dispatches Isabelle/HOL goals to external ATPs (Vampire, E, Z3, CVC4) and reconstructs the proof. There is no direct Lean 4 equivalent. `pythia` addresses the statistics-domain case via `z3_check` for linear-real-arithmetic and Aristotle as a structural oracle. A general-purpose Lean Sledgehammer is out of scope for this paper.

CoqHammer. CoqHammer (Czajka & Kaliszyk, 2018) dispatches Coq goals to external ATPs with kernel-checked reconstruction. `z3_check` in `pythia` adopts the same reconstruction template (witness from external prover, in-kernel checking via `linarith`). CoqHammer is the design template. We do not claim a Lean-side reimplementaion of CoqHammer’s full premise selection.

Mathlib’s existing tactic suite. Mathlib supplies `linarith`, `polyrith`, `nlinarith`, `positivity`, `measurability`, `bound`, `gcongr`. None of these is a statistics-domain hammer. `stats_ineq` is the closest `pythia` relative of the Mathlib bound family. The `@[stats_ineq]` attribute re-tags lemmas as `@[bound]`, so the mature `bound` dispatch picks them up automatically. `pythia` composes with all of the listed Mathlib tactics. The fall-through chain at stage 2 of the dispatcher combines `simp`, `omega`, `linarith`, and `positivity`. The chain is the explicit composition.

5.2. Per-drafter contribution on a benchmark subset

Table 2 reports four FormalAVS problems (Anonymous, 2026) on which the four drafters in our swarm were each tried under a fixed protocol. Aristotle (Harmonic, 2025) is one session per problem. DeepSeek-Prover-V2 (DeepSeek-AI, 2025) (DSPv2-7B GPTQ-int8, a quantised local-GPU prover) and Opus 4.6 (a generalist API drafter) run pass@5 at $N=5$ on the 48-target slate. The point of this subset is to expose the four contribution regimes the benchmark tail surfaces. None of the drafters dominates across the four rows. Per-problem solve outcomes are kernel-checked. Drafter contributions are shown to illustrate *which* drafter helped close which theorem in the corpus, not to claim any drafter is a ground-truth oracle.

The wider table on the full benchmark subset is in Appendix C.

Problem	Aristotle	DSPv2	Opus 4.6
betting.is_kelly_optimal*	1/1	n/a	n/a
etavector [‡]	1/1	1/5	0/5
c_sharp_ranking	1/1	0/5	0/5
equivalence_break_at_finite_precision	0/3 [†]	0/5	0/5

Table 2. Illustrative four-problem subset of FormalAVS. Each cell is (closures)/(attempts). Aristotle is one session per problem. DSPv2 and Opus are pass@5 at $N=5$. The four rows expose four distinct solver regimes. Universal closure sits at the top. Majority closure is etavector, the $\sqrt{2}$ identity that Kimi and Mistral also close at $N=5$. Aristotle-monopoly is c_sharp_ranking. The solver-unreached frontier is equivalence_break. *Companion-archive entry, not in the 48-slate drafter sweep, included for the universal regime. [†]Aristotle returned a counterexample on the as-stated form (missing symmetry hypothesis). The corrected form remains open. [‡]Lean lemma name shortened from the full *etavector_eq_sqrt_two_mul_etahr*, see Anonymous (2026). Full per-tier benchmark numbers in Anonymous (2026).

5.3. Limitations

Three concrete gaps. (i) Domain scope: pythia is statistics-domain only. It does not address general proof automation in Lean 4. (ii) Toolchain pin: the library ships pinned to Lean 4.28 + Mathlib v4.28.0. Mathlib evolves quickly. Each major Mathlib release typically requires a porting pass. (iii) Aristotle dependency: the hardest closures route to a proprietary external prover. The z3_check oracle is open-source. The Aristotle oracle is not. The library is usable without Aristotle for any goal that closes via the pythia dispatcher chain or z3_check. A subset of structural Mathlib gaps requires either Aristotle or a hand proof.

References

Anonymous. FormalAVS: a benchmark for statistical theorem proving in Lean 4, 2026. Companion dataset paper, anonymized for double-blind review.

Blanchette, J. C., Böhme, S., and Paulson, L. C. Extending Sledgehammer with SMT solvers. In *23rd International Conference on Automated Deduction (CADE)*, pp. 116–130, 2011. doi: 10.1007/978-3-642-22438-6\11.

Czajka, Ł. and Kaliszyk, C. Hammer for Coq: Automation for dependent type theory. *Journal of Automated Reasoning*, 61(1-4):423–453, 2018. doi: 10.1007/s10817-018-9458-4.

de Moura, L. and Bjørner, N. Z3: An efficient SMT solver. In *14th International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, pp. 337–340, 2008. doi: 10.1007/978-3-540-78800-3\24.

de Moura, L. and Ullrich, S. The Lean 4 theorem prover and programming language. In *28th International Conference*

on Automated Deduction (CADE), pp. 625–635, 2021. doi: 10.1007/978-3-030-79876-5\37.

DeepSeek-AI. DeepSeek-Prover-V2: advancing formal mathematical reasoning via reinforcement learning for subgoal decomposition, 2025. URL <https://arxiv.org/abs/2504.21801>.

Harmonic. Aristotle: an automated theorem prover for Lean. <https://harmonic.fun/>, 2025. Proprietary commercial tool.

Howard, S. R., Ramdas, A., McAuliffe, J., and Sekhon, J. Time-uniform, nonparametric, nonasymptotic confidence sequences. *Annals of Statistics*, 49(2):1055–1080, 2021. doi: 10.1214/20-AOS1991.

Lieb, E. H. Convex trace functions and the Wigner-Yanase-Dyson conjecture. *Advances in Mathematics*, 11(3):267–288, 1973. doi: 10.1016/0001-8708(73)90011-X.

Limperg, J. and From, A. H. Aesop: White-box best-first proof search for Lean. In *Proceedings of the 12th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP)*, pp. 253–266, 2023. doi: 10.1145/3573105.3575671.

Paulson, L. C. and Blanchette, J. C. Three years of experience with Sledgehammer, a practical link between automatic and interactive theorem provers. In *Proceedings of the 8th International Workshop on the Implementation of Logics (IWIL)*, 2010.

Ramdas, A., Grünwald, P., Vovk, V., and Shafer, G. Game-theoretic statistics and safe anytime-valid inference. *Statistical Science*, 38(4):576–601, 2023.

Rockafellar, R. T. and Uryasev, S. Optimization of conditional value-at-risk. *Journal of Risk*, 2(3):21–41, 2000.

Shin, J., Ramdas, A., and Rinaldo, A. E-detectors: a non-parametric framework for sequential change detection. *New England Journal of Statistics in Data Science*, 2024.

The Mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP)*, pp. 367–381, 2020. doi: 10.1145/3372885.3373824.

The Mathlib Community. Mathlib: the Lean 4 mathematical library. <https://github.com/leanprover-community/mathlib4>, 2025. Repository tagged at v4.28.0 for the pythia pin.

Tropp, J. A. User-friendly tail bounds for sums of random matrices. *Foundations of Computational Mathematics*, 12(4):389–434, 2012. doi: 10.1007/s10208-011-9099-z.

Waudby-Smith, I. and Ramdas, A. Estimating means of bounded random variables by betting. *Journal of the Royal Statistical Society Series B*, 86(1):1–27, 2024.

Whitehouse, J., Ramdas, A., Wu, Z. S., and Sutton, R. Time-uniform self-normalized concentration for vector-valued processes. 2025.

Wold, H. *A Study in the Analysis of Stationary Time Series*. Almqvist & Wiksell, Stockholm, 1938.

A. Library inventory

The shipped library at the AI4MATH 2026 submission cut spans the modules below. Each line lists module path and one-line summary. All listed modules are axiom-clean against $\mathcal{A}_{\text{core}} = \{\text{propext}, \text{Classical.choice}, \text{Quot.sound}\}$ unless explicitly flagged as scaffold.

A.1. Anytime-valid inference

- `Pythia.HowardRamdasCS`: Howard-Ramdas Ville bound for self-normalised confidence sequences (rate $\eta_{\text{HR}}(b) = \sqrt{b \log 2}$). Axiom-clean.
- `Pythia.BettingCS`: betting-CS coverage certificate (rate $\eta_{\text{betting}}(b) = 1/\sqrt{b \log 2 + 1}$). Axiom-clean.
- `Pythia.BettingStrategy`: betting-strategy admissibility scaffolding for the anytime-valid dispatch.
- `Pythia.VilleSupermartingale`: countable-time and finite-horizon Ville inequalities. The `ville_supermartingale`, `ville_ineq`, and `ville_supermartingale.finite` lemmas are the workhorse.
- `Pythia.EDetector`: e-process formalism, with `combine_eprocesses.avg` (average-merge) as the workhorse closer. Anytime-valid Type-I error control via Markov-at-stopping.

A.2. Concentration

- `Pythia.SubGaussianMG`: sub-Gaussian martingale concentration. The Chernoff exponent $\exp(-\tau^2/(2\sigma^2 N))$ bound. Axiom-clean.
- `Pythia.SubGamma`: sub-gamma tail bound. Axiom-clean.
- `Pythia.MGFBoundedSubGamma`: bounded-RV Bernstein-Bennett MGF bound, $\mathbb{E}[\exp(\lambda X)] \leq \exp(\sigma^2 \lambda^2 / (2(1 - b\lambda/3)))$ for centred $|X| \leq b$. Axiom-clean.
- `Pythia.Bernstein`: iid and martingale Bernstein applications.

A.3. Matrix concentration

- `Pythia.MatrixLieb`: Lieb concavity step (Lieb, 1973) and operator-monotone log composition. Scaffold. Dependent operator-trace inequalities route to Aristotle.
- `Pythia.MatrixBernstein`: iid matrix-Bernstein head theorem (Tropp, 2012). Scaffold.
- `Pythia.MatrixBernsteinFull`: martingale extension. Scaffold.

A.4. Other modules

- The `Pythia.Control` subpackage carries `LyapunovDiscrete`, the discrete-time Lyapunov stability result for linear systems.
- `Pythia.Queueing.ErlangB`: Erlang-B blocking formula.
- `Pythia.Queueing.LittlesLaw`: Little’s law steady-state identity.
- `Pythia.TimeSeries.WoldDecomposition` (Wold, 1938).
- `Pythia.Risk.CVaR` (Rockafellar & Uryasev, 2000): the Rockafellar-Uryasev variational characterisation of CVaR.
- `Pythia.Asymptotics.DeltaMethod`, `Pythia.Asymptotics.DeltaMethodMulti`: scalar and multivariate delta method.
- `Pythia.InfoTheory.BretagnolleHuberBinary`: binary-case total-variation bound via the Bhattacharyya bridge.
- The `Pythia.MeasureTheory` subpackage carries `OptionalStoppingUnbounded` and `PathMeasureRN`. Both are structural support for the Ville chain.

B. Worked examples

Each example below is a literal regression test from the `pythia` test suite. Every example compiles axiom-clean against $\mathcal{A}_{\text{core}}$.

B.1. anytime_valid: Ville-bound closer

The countable-time variant closes the canonical Ville maximal inequality on a non-negative supermartingale.

```
import Pythia.Tactic.AnytimeValid
open MeasureTheory ProbabilityTheory
  ENNReal Pythia
```

```
example {Omega : Type*} {m0 :
  MeasurableSpace Omega}
```

```
{mu : Measure Omega} [IsFiniteMeasure mu
]
{f : Nat -> Omega -> Real}
{F : Filtration Nat m0}
(hsup : Supermartingale f F mu)
(hnn : forall t omega, 0 <= f t omega)
(hint : Integrable (f 0) mu)
{c : Real} (hc : 0 < c) :
mu {omega | exists t : Nat, f t omega >=
c}
<= (Integral.integral mu (f 0)).
toNNReal / c.toNNReal := by
anytime_valid
```

The tactic dispatches via the `ville.supermartingale` exact path. Hypotheses are discharged by assumption. No registered rule is consulted. The kernel-checked term reduces under `{propext, Classical.choice, Quot.sound}`.

B.2. z3_check: linear-real-arithmetic with reconstruction

A three-step linear chain over $\mathbb{R}$. Z3 is queried as a ranking filter. `linarith` produces the actual proof term.

```
import Pythia.Tactic.Z3Check
open Pythia
```

```
example {a b c : Real} (h1 : a < b) (h2 : b
<= c) :
a < c := by
z3_check
```

Trace: the encoder produces the SMT-LIB query `(set-logic QF_LRA) ... (assert (< v_a v_b)) (assert (<= v_b v_c)) (assert (not (< v_a v_c))) (check-sat)`, which Z3 returns `unsat`. `linarith` then constructs the Farkas certificate inside Lean. Z3 cannot smuggle axioms. If `linarith` disagreed, the tactic would fail loud.

B.3. pythia: chain-rule fall-through to aesop

A goal with no matching `@[stat.lemma]` but a closed-form chain in `linarith`. Stage 1 (aesop with the Pythia ruleset) declines. Stage 2 (the cleanup chain combining `simp`, `omega`, `linarith`, and `positivity`) closes.

```
import Pythia.Tactic.Pythia
open Pythia
```

```
example (a b c d : Real)
(h1 : a <= b) (h2 : b <= c) (h3 : c <= d
) :
a <= d := by
pythia
```

The dispatcher's first-combinator tries `aesop`, observes no applicable `@[stat.lemma]` closer, then tries the cleanup chain. `linarith` closes the chain in one step.

B.4. Registry-extend: tagging a user theorem

User extensions consist of tagging library lemmas with the appropriate attribute. Below: a trivial real-arithmetic identity registered for `pythia` via `@[stat.lemma]`.

```
import Pythia.Tactic.Pythia
open Pythia

@[stat_lemma]
theorem add_zero_real (x : Real) : x + 0 =
x := by ring

-- pythia now closes goals matching the
-- registered lemma:
example (x y : Real) : (x + 0) + (y + 0) =
x + y := by
pythia
```

The attribute re-elaborates as `@[aesop safe apply (rule_sets := [Pythia])]`. The lemma then joins the Pythia aesop ruleset and is consulted by `pythia` on every subsequent invocation. No fork or rebuild of the tactic implementation is required.

C. Aristotle vs DSPv2 vs Opus 4.6 (wider subset)

Table 3 reports a wider subset of FormalAVS (Anonymous, 2026) per-tier closure rates for the three solver families. Aristotle is one-attempt-per-target. DSPv2 and Opus are pass@5 at $N=5$ on the 48-target slate (T0 through T2 plus T3). Cells are *(closures)/(targets)* at the tier denominator, with T4 (library-gap) and T5 (capability-ceiling) appended.

Solver	T0	T1	T2	T3	T4	T5
Aristotle	7/7	33/34	7/8	6/6	0/4	0/1
DSPv2-7B GPTQ-int8	3/7	8/34	0/8	1/6	0/4	0/1
Opus 4.6	0/7	13/34	0/8	0/6	0/4	0/1

Table 3. Wider per-tier comparison from FormalAVS. Aristotle: one attempt per target. DSPv2 and Opus: pass@5 at $N=5$. T4 is solver-hit-Mathlib-gap (every solver 0/4 for the same library-gap reason). T5 is the all-solver capability-ceiling target. The 6/6 vs 1/6 vs 0/6 column at T3 (cross-function inequality wall) is the benchmark's cleanest solver-specific signal. The single DSPv2 T3 closure is the $\eta_{\text{VECTOR}} = \sqrt{2} \eta_{\text{HR}}$ identity (the *etavector* target). Kimi K2.5 and Mistral Large 3 also close this one in the wider drafter sweep. Source: dataset paper (Anonymous, 2026), Tables 1-2 reproduced verbatim.

Per-target capability-wall tail. Table 4 reports the per-target closure matrix on the five capability-wall tail problems used in the dataset paper to classify solver regimes.

Target	Aristotle	Kimi K2.5	Sonnet 4.6	Gemini 3 Pro
<code>betting_is_kelly_optimal*</code>	1/1	1/1	1/1	1/1
<code>c_vector_eq_sqrt_two_mul_c_HR</code>	1/1	1/1	0/5	1/2
<code>c_sharp_ranking</code>	1/1	0/9	0/5	1/3
<code>quantized_entropy_bound</code>	1/1	0/9	0/5	0/3
<code>equivalence_break_at_finite_precision</code>	0/3 [†]	0/19	0/15	0/13

Table 4. Per-target capability-wall tail. Cells are *(closures)/(attempts)*. Five distinct solver regimes appear: universal closure, majority closure, drafter-monopoly (Gemini’s `gcongr-prior` closes `c_sharp_ranking`), Aristotle-monopoly, and solver-unreached. Source: Anonymous (2026), Table 4. *Companion-archive entry, not in the 48-slate drafter sweep. [†]Aristotle: two sessions timed out at the 20-minute compute gate. One returned a counterexample on the as-stated form (missing symmetry hypothesis, (Anonymous, 2026) §Aristotle-refute).

D. Reproduction

Toolchain pin. Lean 4.28 + Mathlib v4.28.0. The repository’s `lean-toolchain` file declares the exact toolchain. The Mathlib dependency in `lakefile.toml` pins the v4.28.0 release tag.

Build. The library URL is redacted for double-blind review (full URL in camera-ready). Reproduction commands once the URL is supplied:

```
git clone <pythia-url-redacted>
cd pythia
lake exe cache get
lake build
```

Axiom audit. Each shipped module ends with a `#print axioms` block. The expected audit set is `{propext, Classical.choice, Quot.sound}`. Any deviation fails the audit.

FormalAVS benchmark. The full benchmark is reported in Anonymous (2026). Reproduction commands for the benchmark are in that paper’s reproduction appendix. The benchmark URL is redacted for double-blind review.

E. Related work, extended

E.1. Lean 4 tactic infrastructure

Mathlib ships a tactic suite the `pythia` dispatcher composes with. `linarith` produces Farkas certificates and `polyrith` routes polynomial-ideal goals via Sage with kernel-checked reconstruction. `nlinarith` extends `linarith` to a small non-linear fragment. Sign goals route through `positivity` and σ -algebra-membership goals through `measurability`. Generalised-congruence monotonicity is discharged by `gcongr`. Monotone-composition goals route through `bound`, which uses an attribute-driven rule database. The closest Mathlib relative to `pythia` is `bound`: the `@[stats.ineq]` attribute literally re-tags lemmas as `@[bound]`, so the underlying `bound` dispatch picks them up automatically. `Aesop`

(Limperg & From, 2023) supplies the `rule_sets` mechanism the `pythia` dispatcher uses. `pythia` is therefore a thin domain-registration layer over `aesop` plus a Z3 oracle.

E.2. Sledgehammer (Isabelle)

Sledgehammer (Paulson & Blanchette, 2010; Blanchette et al., 2011) dispatches Isabelle/HOL goals to Vampire, E, Z3, CVC4, with proof reconstruction inside Isabelle’s kernel via Metis or `smt-tactic` replay. The reconstruction discipline is the template for both `CoqHammer` and `z3_check`. There is no Lean 4 equivalent at the time of writing. `pythia` does not attempt the Isabelle Sledgehammer’s full premise-selection front end. The library ships only the linear-real-arithmetic instance via `z3_check`.

E.3. CoqHammer (Coq)

`CoqHammer` (Czajka & Kaliszyk, 2018) dispatches Coq goals to Vampire, E, Z3, CVC4 with kernel-checked reconstruction. Premise selection in `CoqHammer` is k-NN over a featurised representation of the Coq context. The reconstruction step is a small certifying tactic (`sauto`) that replays the ATP’s witness inside the Coq kernel. `z3_check`’s reconstruction via `linarith` mirrors this template at small scale. The external prover supplies the verdict and the in-kernel tactic supplies the proof term. The kernel is the final arbiter. We do not claim a full Lean-side reimplementaion of `CoqHammer`’s premise-selection layer.

E.4. Statistical formalisation libraries

Mathlib’s `Probability.Martingale` module supplies Doob decomposition and Azuma’s inequality but does not instantiate the canonical confidence-sequence families, the per-family rate functions, or Ville’s inequality for the betting construction. The `pythia` library closes that gap. The FormalAVS benchmark (Anonymous, 2026) measures the result. Coq’s `mathcomp.analysis` and the Isabelle/HOL probability formalisations cover overlapping but disjoint sub-areas, and are not attribute-driven hammers.